Lecture 2-3 — DOM Style Manipulation

Introduction

This lesson teaches how to change CSS styles from JavaScript. We'll cover **element.style**, camelCase property names, **getComputedStyle**, and event listeners, using a small interactive demo.

Live demo — change background color and font size

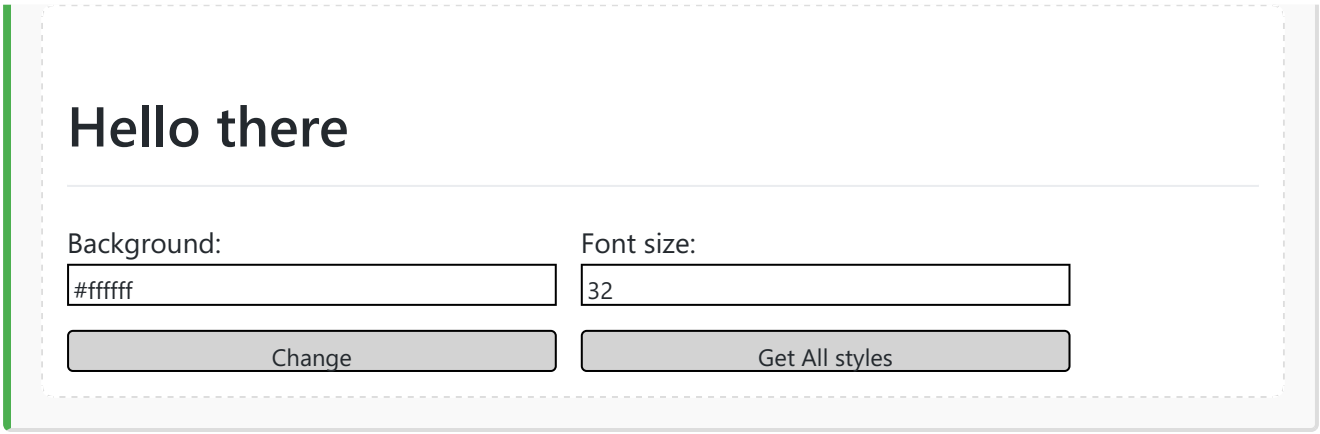
```
<input type="color" id="col">
<input type="number" id="fsize" min="8" max="72">
<button id="mybtn">Change</button>
<h1 id="special">Hello there</h1>

// JS
special.style.backgroundColor = col.value;
special.style.fontSize = fSize.value + 'px';

// read computed styles
getComputedStyle(special).transition
getComputedStyle(special).fontFamily
```

Output:

Powered by SPRK Technologies



Detailed explanation for beginners

- **CSS vs JS names:** In CSS you write `background-color`. In JavaScript you use camelCase: `element.style.backgroundColor`.
- **Inline style object:** `element.style` reflects styles set directly on the element (inline styles). Reading `element.style.backgroundColor` only shows values set via JS or the style attribute.
- **Computed styles:** Use `getComputedStyle(element)` to read the final style the browser applies (including styles from stylesheets, classes, and user agent defaults).
- **Units matter:** When setting sizes, always add units (e.g., `px`) unless the property accepts unitless numbers.
- **Event listeners:** Use `addEventListener` to attach behavior. To remove a listener later you must call `removeEventListener` with the same function reference.

Common mistakes and tips

- Don't try to subtract CSS names like `background - color` — use camelCase.
- To toggle visual states, prefer adding/removing CSS classes and control styles in CSS (keeps JS simpler).
- Use `parseInt` or `Number()` to validate numeric inputs before appending units.
- Remember that styles from external stylesheets won't appear in `element.style` — use `getComputedStyle` to inspect them.

Powered by SPRK Technologies

Extra example — toggle a CSS class (recommended approach)

```
// CSS
.highlight { background-color: yellow; color: black; }

// JS
special.classList.toggle('highlight');
```

Output:

Toggle highlight

Target text

Summary

To change how elements look from JavaScript: prefer toggling classes for maintainability, use `element.style` for quick inline changes, and use `getComputedStyle` when you need the final computed values. Always validate inputs and remember to use the correct property names.

End of Lecture 2-3

Powered by SPRK Technologies